Generalized Sorting with Predictions

Pinyan Lu * Xuandi Ren † Enze Sun ‡ Yubo Zhang §

August 19, 2020

Abstract. Generalized sorting problem, also known as sorting with forbidden comparisons, was first introduced by Huang et al. [HKK11] together with a randomized algorithm which requires $\tilde{O}(n^{3/2})$ probes. We study this problem with additional predictions for all pairs of allowed comparisons as input. We propose a randomized algorithm which uses $O(n \log n + w)$ probes with high probability and a deterministic algorithm which uses $O(nw)$ probes where w is the number of mistakes made by prediction.

1 Introduction

Sorting is arguably the most basic computational task, which is also widely used as an important component of many other algorithms. Pair-wise comparison is the heart of most sorting algorithms. In the standard model, it is assumed that we can do comparison between all pairs of elements as we want and they are all of the same cost. However, this may not be the case in many applications. There might be some feasibility constrains which forbid us to compare some pairs of elements, or the costs for comparing different pairs may be different.

In this paper, we focus on the feasibility constrains, which is a special case of the non-uniform cost model. We can view the elements as the vertices of a graph and allowed comparison pairs as the edges of the graph. Then, the generalized sorting problem can be viewed as the following graph problem. The input is an undirected graph $G = (V, E)$ where V is the set of elements to be sorted and E defines the set of allowed comparisons. It is promised that the hidden directed graph is acyclic and contains a Hamiltonian path. Our algorithm adaptively probes the direction of edges in E to find a Hamiltonian path. We measure the complexity of an algorithm by the number of edges it probes. In the standard setting, where G is a complete graph, the minimum number of probes required is $\Theta(n \log n)$. For general graphs, one may need more comparisons. Huang, Kannan and Khanna has proved an upper bound of $\tilde{O}(n^{1.5})$ by giving a randomized algorithm [HKK11].

Recently, there is an interesting line of research called algorithm design with predictions which is motivated by the observation that machine learning can help algorithm design. For example, let us consider the classic binary search algorithm, which can find the position of an existing element in a sorted list in $O(\log n)$. It starts by querying the median of the list. However, if a machine learning algorithm can roughly estimate the quantile of the given element. It may not be always a good idea to start from the middle. Based on this idea, one designed an algorithm with query complexity of $O(\log w)$, where w is the distance between the true quantile and the estimated one. This could be much better than $O(\log n)$ when w is much smaller than n.

In this paper, we initiate the study of generalized sorting with predictions. The model is very natural. Besides the undirected graph $G = (V, E)$, we are also given an orientation prediction of the edges, which is an estimation of the hidden directions. The number of mis-predicted edge is denoted by w. With the help from the predictions, we hope to improve the bound of $\tilde{O}(n^{1.5})$ when w is small. We provide a randomized algorithm for the generalized sorting problem and prove that the number of probes it makes is $O(n \log n + w)$ with high probability. The description of the algorithm is very simple while its analysis is

*Shanghai University of Finance and Economics, lu.pinyan@mail.shufe.edu.cn
†Peking University, 1800012911@pku.edu.cn
‡Shanghai Jiao Tong University, sun_en_ze@sjtu.edu.cn
§Peking University, zhangyubo18@pku.edu.cn

quite subtle and involved. We also present a deterministic algorithm which uses $O(nw)$ probes to show that the generalized sorting problem can be solved using only linear probes when w is as small as a constant.

1.1 High-level idea of the algorithm

In a high level, we want to determine the direction of (predicted) incoming edges of a vertex u with the smallest number of probes. In the ideal case, when we know a total order of v's in-neighbors, we can probe the edge from its largest in-neighbor to it. If the prediction is correct, we can conclude that all its in-edges in prediction are correct and do not need to probe other edges anymore. If the prediction is incorrect, we identify a wrong prediction and move on to probe the second largest until we verify the first correct edge in this order. The wrong probes are charged to the w term of complexity. In the algorithm, we maintain a set of vertices A whose incoming edges in prediction are already determined. If there is a vertex u whose in-neighbors have a total order in A, we can probe its incoming edges in the above order and then put it into A. One hidden observation is that when the first k elements are in A, there must be a total order among them. Then the $(k+1)$-th element will be in the ideal case whose in-neighbors have a total order if all predictions are correct. This is how the algorithm makes progress.

However, due to the mistakes in prediction, we may not be able to find such a vertex. Then we need to do some probes to correct some edges so that such a vertex appears. Of course, we cannot probe all the edges. The next important observation is that if we probe and verify one in-neighbor outside A or two in-neighbors in A whose order is not determined yet for a vertex u, we know for sure that u is not the vertex we are looking for. We call this vertex or this pair of vertices a certificate for u. The idea is that we do not need to probe more incoming edges of a vertex who has a certificate. This allows us not to probe too many edges. In this way, by probing at most two incoming edges of each vertex (if we figure out a mis-prediction edge, we move on to a new edge and charge that probe to the term w), we should be able to find out a new vertex to be put into A. Moreover, if a vertex already has a valid certificate, we do not need to probe any of its incoming edges. Since everything else is linear in n or w, the complexity term $O(n \log n)$ comes from the probes for certificates. We need to find a new certificate for a vertex when its previous one becomes invalid (i.e. the in-neighbor is added into A or the order of the two in-neighbors is determined). In the worst case, one may need to update the certificates again and again and probe too many edges. Then, we have the idea of choosing a random certificate to avoid the worst case, which also complicates the analysis. In the analysis, we make some important observations about the stochastic process given by the randomized algorithm and prove the complexity of $O(n \log n)$.

1.2 Related works

One special case of the generalized sorting problem is the problem of matching nuts and bolts [ABF+94, KMS98], where the graph is a complete bipartite graph. The model of general comparison graphs was introduced in Huang et al. [HKK11]. The non-uniform cost model is studied in [CFG+02, GK01, Kha03].

Algorithms with predictions are studied for caching [LV18, Roh20], ski-rental and online scheduling [PSK18] and other problems. See a nice survey by Mitzenmacher and Vassilvitskii [MV20].

2 Preliminaries

The input to the generalized sorting problem with prediction is an undirected graph $G = (V, E)$ together with an orientation $\vec{P}$ of E, where $\forall (u, v) \in E$, either $(u, v) \in \vec{P}$ or $(v, u) \in \vec{P}$ holds. We call $\vec{P}$ the (directed) edges in prediction. There is another orientation $\vec{E}$ of E which is unknown to us. Let $\vec{G}_P = (V, \vec{P})$ and $\vec{G} = (V, \vec{E})$, it's guaranteed that $\vec{G}$ is acyclic and there is a directed Hamiltonian path in $\vec{G}$. Our goal is to adaptively probe the direction of edges in $\vec{E}$ and find out the underlying Hamiltonian path. The performance of an algorithm is measured by the number of edges it probes, which may be in terms of n and w, where $n = |V|$ is the number of vertices and $w = |\vec{P} \backslash \vec{E}|$ denotes the number of mis-predicted edges.

Since the graph $\vec{G} = (V, \vec{E})$ is acyclic, any subset $\vec{E}' \subseteq \vec{E}$ naturally defines a total order $(V, <_{\vec{E}'})$, where $a <_{\vec{E}'} b$ denotes there is a directed path from a to b which only consists of edges in $\vec{E}'$.

An algorithm may adaptively probe some edges. For convenience, we use $\vec{Q}$ to denote the set of already probed (directed) edges when discussing. For every vertex u, denote by S_u its incoming vertices in both $\vec{G}$ and $\vec{G}_P$, i.e. $S_u = \{v|(v,u) \in \vec{P} \cap \vec{E}\}$, and denote by T_u its incoming vertices v in $\vec{G}_P$ s.t. (v,u) is correct or unprobed, i.e. $T_u = \{v|(v,u) \in \vec{P} \wedge (u,v) \notin \vec{Q}\}$. Notice that T_u is always known to us while S_u is not. We always have $S_u \subseteq T_u$ since $\vec{Q} \subseteq \vec{E}$.

Let S be a vertex set, denote by $G[S]$ the subgraph induced in G by S, i.e. the graph $(S, \{(a,b)|a,b \in S \wedge (a,b) \in E\})$.

3 An algorithm using $O(n \log n + w)$ probes

3.1 Description

In our algorithm we maintain a vertex set A. We make sure that when a vertex u is added into A, all its incoming edges in prediction must have been determined, i.e. $\forall (v,u) \in \vec{P}, v <_{\vec{Q}} u \vee u <_{\vec{Q}} v$ holds. Therefore all edges in the induced subgraph $G[A]$ are always determined. We denote by $(A, <_A)$ the partial order defined on the induced subgraph $\vec{G}[A]$, i.e. $a <_A b$ means there is a directed path from a to b which only passes vertices in A. $a \not<_A b$ means there is no such a path.

We initialize A as $\emptyset$, then iteratively add active vertices to A and maintain $<_A$ accordingly until $A = V$ and we find the desired total order.

Definition 1 (active vertex)**.** We call $u \in V$ an active vertex if both of the following conditions are satisfied:

1. $S_u \subseteq A$
2. There is a total order of S_u in $(A, <_A)$.

Lemma 2. *There is always at least an active vertex if $A \neq V$.*

Proof. Suppose the Hamiltonian path in $\vec{G}$ is $(a_1, ..., a_n)$. Let k be the smallest number s.t. $a_k \notin A$ ($k \leq n$ since $A \neq V$), then a_k satisfies $S_{a_k} \subseteq \{a_1, ..., a_{k-1}\} \subseteq A$ and there must be a total order of S_{a_k} in $(A, <_A)$. Therefore a_k is active at the moment. □

What we want to do is to iteratively add an active vertex into A. Since $\vec{E}$ and S_u is unknown to us, we can not directly know whether a vertex u is active or not. But for those which are not active currently, we can find a 'certificate' to prove that it is not active. There are two types of certificates, which are defined as follows:

Definition 3 (certificate)**.**

1. A type-1 certificate for u is a vertex $v \in S_u$ s.t. $(v,u) \in \vec{Q} \wedge v \notin A$.
2. A type-2 certificate for u is a pair of different vertices $v_1, v_2 \in S_u$ s.t. $(v_1,u), (v_2,u) \in \vec{Q} \wedge v_1, v_2 \in A \wedge v_1 \not<_A v_2 \wedge v_2 \not<_A v_1$.

Notice a vertex is active if and only if no certificate can be found for it.

A valid certificate for u may become invalid since A will be extended. In such cases we need to look for a new one for it.

For a vertex u which is not determined to be active or not, we try to find a certificate for it by probing edges from T_u to u.

If $T_u \not\subseteq A$, there may be a type-1 certificate $v \in T_u \backslash A$ for u as long as the direction of (v,u) is correct and $v \in S_u$ actually. We randomly choose such a v and probe the edge (v,u).

If $T_u \subseteq A$ but there are two different vertices $v_1, v_2 \in T_u$ s.t. $(v_1 \not<_A v_2) \wedge (v_2 \not<_A v_1)$, (v_1, v_2) may become a type-2 certificate for u as long as the directions of $(v_1,u), (v_2,u)$ are both correct and $v_1, v_2 \in S_u$. We also randomly choose such a pair (v_1, v_2), probe (v_1,u) and (v_2,u).

If it is in one of the above cases, we either find a mis-predicted edge, or find a valid certificate for u to prove it is not an active vertex now. Otherwise we have $T_u \subseteq A$ and there is a total order in $(T_u, <_A)$, u

must have been active since $S_u \subseteq T_u$. We only need to determine its incoming edges in prediction. This can be done by iteratively probing the edge from the largest vertex in T_u to u until we have found a correct edge or T_u has become empty. Now we can deduce the remaining edges from T_u to u must be all correct by transitivity. Hence u can be safely added into A now.

The pseudo code of our algorithm is stated as follows:

Algorithm 1 Randomized algorithm using $O(n \log n + w)$ probes

```
set A := ∅
while A ≠ V do
    pick u ∈ V\A with the smallest label s.t. u does not have a valid certificate
    if T_u ⊈ A then
        randomly pick v ∈ T_u\A
        probe (v, u)
    else if ∃ different v_1, v_2 ∈ T_u s.t. (v_1 ≮_A v_2) ∧ (v_2 ≮_A v_1) then
        randomly select such a pair v_1, v_2
        probe (v_1, u), (v_2, u)
    else
        let v be the largest vertex in (T_u, <_A)
        probe (v, u)
        if the direction of (v, u) is correct then
            add u to A and update <_A
```

3.2 Analysis

Recall that the edges in $G[A]$ are always determined. Therefore when $A = V$ and the algorithm terminates, we can always get the desired total order. What we need to prove is that the algorithm always terminates normally.

Theorem 4. *The algorithm always terminates in finite steps.*

Proof. In each loop, exactly one of the following things happens:

- we find a mis-predicted edge
- we add a vertex u into A
- we find a certificate for u who doesn't have a valid one previously

The first case happens for finite times because once we find a mis-predicted edge, we will expel a vertex from T_u.

Each vertex is added into A for at most once, so the second case happens for finite times.

When the third case happens, we find a new certificate for u. When it becomes invalid, the number of valid certificates for u will decrease by at least one. Therefore this case also happens for finite times. □

We have shown the algorithm always ends up with the desired total order in finite steps. Now we present some lemmas to help analyze the number of probes made by our algorithm.

Lemma 5. *All vertices are added into A in a fixed order regardless of the randomness.*

Proof. Notice we only add active vertices into A, and whether a vertex u is active or not only depends on the current A. Therefore $\forall 1 \leq i \leq n$, when $|A| = i - 1$, the i-th vertex added into A is always the one with the smallest label among all active vertices regardless of randomness. □

Corollary 6. *The order of all pairs of vertices are determined in $<_A$ in a fixed order, regardless of the randomness.*

Remark. Lemma 5 and corollary 6 use the fact that we determine the order among vertices in T_u according to not $<_{\vec{Q}}$ but $<_A$. The algorithm is still correct if we also use $<_{\vec{Q}}$ and seems more efficient since we use more information, but it will create subtle correlation in the stochastic process and we do not know how to analyze. The above fixed order property is crucial in the proof of our main theorem.

Lemma 7. *For a random permutation $Y_1, ..., Y_n$ over $\{1, ..., n\}$, the number of elements Y_i s.t. $Y_i = \max_{j \le i} Y_j$ does not exceed $6 \ln n + 6$ w.p. at least $1 - \frac{1}{2n^2}$.*

Proof. We build the random permutation in such a way: $\forall 1 \le i \le n$, we randomly and independently pick each Z_i from $\{1, ..., i\}$, and take the permutation $Y_1, ..., Y_n$ s.t. $\forall i, Y_i$ is the Z_i-th largest element in $\{Y_1, ..., Y_i\}$. It's easy to see such a permutation is unique, and the one produced in this way is a uniformly random permutation.

Let $X_1, ..., X_n$ be a sequence of 0-1 random variable where X_i indicates whether $Z_i = i$. Notice $Z_i = i \Leftrightarrow Y_i = \max_{j \le i} Y_j$, so the number we mentioned in the lemma is just the sum of all X_i's. Let $X = \sum_{i=1}^n X_i$, we have

$$\Pr[X_i = 1] = 1 - \Pr[X_i = 0] = \frac{1}{i}$$

Therefore $\mathbb{E}[X] = H_n = \sum_{i=1}^n \frac{1}{i} \approx \ln n + \gamma$ as $n \to \infty$ where γ is the euler constant, and we have

$$\ln n + \gamma < H_n \le \ln n + 1, \forall n \ge 1$$

Notice all X_i's are independent, from Chernoff bound we have

$$\Pr[X > (1+\epsilon)\mu] \le \exp\left(-\frac{\mu\epsilon^2}{2+\epsilon}\right)$$

take $\epsilon = 5$ we have

$$\begin{aligned}\Pr[X > 6(\ln n + 1)] &\le \exp\left(-\frac{25}{7}(\ln n + \gamma)\right) \\ &\le \frac{1}{2n^2}\end{aligned}$$

□

Theorem 8. *With probability at least $1 - \frac{1}{n}$, the algorithm uses no more than $O(n \log n + w)$ probes, where $w = |\vec{P} \backslash \vec{E}|$ is the number of wrong edges in the prediction.*

Proof. The wrong probes are charged to the w term of complexity, so we only need to prove that w.p. $1 - \frac{1}{n}$, the total number of correct probes in line 6, line 9 and line 12 does not exceed $O(n \log n)$.

Once a correct probe is made in line 12, u will be added into A. Since each vertex is added into A only once, the number of correct probes here is at most n.

Each correct probe in line 6 and line 9 leads to a valid certificate for u. We won't make any further probes for u until its current certificate becomes invalid.

According to lemma 5, all vertices are added into A in a fixed order, this means all initially valid type-1 certificates for u, which is just the set S_u, will become invalid in a fixed order $s_1, ..., s_{|S_u|}$.

In our analysis, it can be viewed as we have taken a random permutation over all possible type-1 certificates at first, instead of randomly selecting the next one when needed. For those $s_i \in S_u$ which has been added into A and is no longer a valid type-1 certificate for u when considered, we simply ignore it so that it makes no influence. Suppose S_u is permuted as $s_{Y_1}, s_{Y_2}, ..., s_{Y_{|S_u|}}$, i.e. the i-th element lies in the Y_i-th position of the order they become invalid, the number of valid type-1 certificates taken into use is just the number of elements Y_i s.t. it is the maximum in $\{Y_1, ..., Y_i\}$. From lemma 7 we know w.p. at least $1 - \frac{1}{2n^2}$, this number does not exceed $6 \ln n + 6$, take union bound over all u's, we have w.p. at least $1 - \frac{1}{2n}$, no vertex uses more than $(6 \ln n + 6)$ type-1 certificates, hence the correct probes in line 6 does not exceed $6n \ln n + 6n$.

The above analysis is exactly the same for type-2 certificates, since according to corollary 6, the possible type-2 certificates for u will also become invalid in a fixed order. The only difference is that the number

of valid type-2 certificates for each u may be up to n^2, while it is at most n for type-1 certificates. Again use lemma 7 and take union bound, w.p. at least $1 - \frac{1}{2n}$, the correct probes in line 9 does not exceed $12n \ln n + 6n$.

Take union bound over the two cases and we can conclude finally that w.p. at least $1 - \frac{1}{n}$, the algorithm uses no more than $O(n \log n + w)$ probes in total.

□

4 An algorithm using $O(nw)$ probes

Here we briefly introduce a deterministic algorithm which requires $O(nw)$ probes to show that the generalized sorting problem can be solved using only linear probes when w is as small as a constant.

The basic idea is to find a mis-predicted edge in $O(n)$ time and correct it in the predicted graph. We denote by $\vec{P}_C$ the edge set of the predicted graph after correction: $\vec{P}_C = \{(v, u) \in \vec{P} | (u, v) \notin \vec{Q}\} \cup \vec{Q}$.

If there is a directed cycle in $\vec{G}_C = (V, \vec{P}_C)$, there must be some wrong edges on this cycle since $\vec{G}$ is acyclic. We just probe all the edges on the cycle, update $\vec{P}_C$ and loop again.

If $\vec{G}_C$ is acyclic, consider running topological sort on it. If all edges in $\vec{P}_C$ are correct, each time there will be exactly one vertex whose in-degree is 0. Otherwise there may be two vertices v_1, v_2 with in-degree 0 at the same time, and this happens only when there are some mis-predicted edges adjacent to v_1, v_2 or on the path produced by topological sort before. We probe all these edges and loop again.

The pseudo code is stated as follows:

Algorithm 2 Deterministic algorithm using $O(nw)$ probes

```
1: while there is no Hamiltonian path in (V, Q⃗) do
2:     if there is a simple directed cycle in G⃗_C then
3:         probe all the edges on the cycle
4:     else
5:         run topological sort on G⃗_C and stop when ∃v_1, v_2 both with in-degree 0
6:         let (a_1, ..., a_k) be the (partial) topological order
7:         ∀1 ≤ i < k, probe the edge (a_i, a_{i+1})
8:         if k ≠ n then
9:             probe all edges adjacent to v_1, v_2
```

Theorem 9. *The algorithm always finds a correct Hamiltonian path in $O(nw)$ probes.*

Proof. There are 3 cases in a loop, we analyze them separately:

1. *There is a simple directed cycle in $\vec{G}_C$.* Since the actual $\vec{G}$ is acyclic, at least one edge on the cycle in $\vec{G}_C$ is mis-predicted. By probing all edges on it we will find at least one mis-predicted edge.

2. *$\vec{G}_C$ is acyclic and $k = n$ in line 8, i.e. the topological sort terminates normally.* By probing all edges between the adjacent vertices in the topological order, we either find a mis-predicted edge, or can confirm that the path is exactly the correct Hamiltonian path.

3. *$\vec{G}_C$ is acyclic and there are two different vertices v_1, v_2 with in-degree 0 during the topological sort.*

   We claim that we will find a mis-predicted edge in line 7 or line 9, and prove it by contradiction. Assume all edges probed in line 7 and line 9 are correct, if $k > 0$, (a_k, v_1) and (a_k, v_2) must lie in $\vec{G}_C$ since the topological sort proceeds normally for $a_1, ..., a_k$, and $\forall 1 \leq i \leq k, (a_i <_{\vec{E}} v_1) \wedge (a_i <_{\vec{E}} v_2)$ must hold due to transitivity. W.l.o.g assume $v_1 <_{\vec{E}} v_2$. Consider the directed path from v_1 to v_2 in the actual graph $\vec{G}$, let it be $(b_1, ..., b_l)$ where $b_1 = v_1$ and $b_l = v_2$. Notice $a_1 <_{\vec{E}} ... <_{\vec{E}} a_k <_{\vec{E}} b_1 = v_1 <_{\vec{E}} ... <_{\vec{E}} b_l = v_2$. Therefore $(b_{l-1}, v_2) \in \vec{G}_C$ and $b_{l-1} \notin \{a_1, ..., a_k\}$, which contradicts the fact that the in-degree of v_2 is 0 at that moment.

Therefore in each loop, we either find at least one mis-predicted edge in $\vec{G}_C$ or find a correct Hamiltonian path. It's obvious that the number of probes we make is $O(n)$ in each loop, so the total number of probes does not exceed $O(nw)$. □

References

[ABF+94] Noga Alon, Manuel Blum, Amos Fiat, Sampath Kannan, Moni Naor, and Rafail Ostrovsky. Matching nuts and bolts. In *SODA*, pages 690–696, 1994.

[CFG+02] Moses Charikar, Ronald Fagin, Venkatesan Guruswami, Jon Kleinberg, Prabhakar Raghavan, and Amit Sahai. Query strategies for priced information. *Journal of Computer and System Sciences*, 64(4):785–819, 2002.

[GK01] Anupam Gupta and Amit Kumar. Sorting and selection with structured costs. In *Proceedings 42nd IEEE Symposium on Foundations of Computer Science*, pages 416–425. IEEE, 2001.

[HKK11] Zhiyi Huang, Sampath Kannan, and Sanjeev Khanna. Algorithms for the generalized sorting problem. In *2011 IEEE 52nd Annual Symposium on Foundations of Computer Science*, pages 738–747. IEEE, 2011.

[Kha03] Sanjeev Khanna. Selection with monotone comparison costs. In *Proceedings of the fourteenth annual ACM-SIAM symposium on Discrete algorithms*, page 10. SIAM, 2003.

[KMS98] János Komlós, Yuan Ma, and Endre Szemerédi. Matching nuts and bolts in $O(n \log n)$ time. *SIAM Journal on Discrete Mathematics*, 11(3):347–372, 1998.

[LV18] Thodoris Lykouris and Sergei Vassilvitskii. Competitive caching with machine learned advice. *arXiv preprint arXiv:1802.05399*, 2018.

[MV20] Michael Mitzenmacher and Sergei Vassilvitskii. Algorithms with predictions. *arXiv preprint arXiv:2006.09123*, 2020.

[PSK18] Manish Purohit, Zoya Svitkina, and Ravi Kumar. Improving online algorithms via ml predictions. In *Advances in Neural Information Processing Systems*, pages 9661–9670, 2018.

[Roh20] Dhruv Rohatgi. Near-optimal bounds for online caching with machine learned advice. In *Proceedings of the Fourteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 1834–1845. SIAM, 2020.